

MinorFlow: A gem5 Pipeline Visualizer for Teaching Computer Architecture^{*}

Manuel Nieto¹[0009–0007–3893–5628], Francisco Cortez Casini¹[0009–0008–0892–0545], María Delfina Vélez Ibarra¹[0000–0001–6768–5313], and Gonzalo Tomás Vodanovic^{1,2}[0009–0001–9650–5446]

¹ Facultad de Matemática, Astronomía, Física y Computación, Universidad Nacional de Córdoba, Córdoba, Argentina

² Facultad Regional Villa María, Universidad Tecnológica Nacional, Villa María, Argentina

`manuel.nieto@mi.unc.edu.ar`

Abstract. Computer architecture is a cornerstone of High-Performance Computing education, reconciling abstract algorithms with underlying hardware constraints. However, a significant pedagogical gap exists between the simplified textbook pipelines taught in undergraduate courses and the complex realities of modern processors. Cycle-level simulators, like gem5, partially bridge this gap by exposing realistic microarchitectural behavior, yet their default text-based statistical outputs fail to convey the temporal dynamics of pipeline execution, leaving students unable to intuitively grasp stalls, cache misses, or branch mispredictions. In order to address this, we introduce MinorFlow, an open-source visualization tool tailored for gem5’s MinorCPU model. A command-line tracer reduces the raw MinorTrace to a compact document, and a single-file viewer renders that document as a multi-cycle pipeline diagram in any browser, with no installation. The visual language follows the row-per-instruction, column-per-cycle convention familiar to students of computer architecture, and exposes events that remain hidden in aggregate statistics: fetch request and response separation, inter-stage buffer stalls, scoreboard decisions, load-store queue activity, and branch redirects with the flushes they cause. Through functional validation and targeted case studies, we show how the tool exposes microarchitectural causal relationships and makes performance bottlenecks visually identifiable. MinorFlow lowers the entry barrier to detailed microarchitectural analysis in gem5, and is released openly for its user community.

Keywords: Pipeline visualization · gem5 · Computer architecture education · RISC-V

1 Introduction

High-Performance Computing (HPC) underpins much of modern scientific and technological progress [5], and computer architecture education is what prepares

^{*} Source code available at: <https://github.com/FaMAF-CVA6-Project/MinorFlow>

the professionals who will work with it, teaching them to reason about cache behavior, branch prediction and memory latency hiding. A widely recognized reference for the subject is Computer Organization and Design by Patterson and Hennessy [15], which relies on a deliberately simplified five-stage in-order pipeline for pedagogical purposes. While effective for introducing foundational concepts, that abstraction leaves a structural gap between the classroom and the processors students later encounter in research or industry.

To bridge this conceptual gap, educators often turn to cycle-level simulators like gem5 [4] to expose students to realistic microarchitectural dynamics beyond textbook abstractions. However, the standard educational practice often reduces this tool to simply running a simulation and inspecting the aggregate metrics it reports at the end of each run. Relying entirely on this flat text summary (typically the `stats.txt` file, listing instructions-per-cycle (IPC), cache hit rates, and cycle counts) renders the exercise fundamentally non-explanatory, leaving dynamic pipeline interactions and execution bottlenecks hidden. Addressing this opacity requires visualization, which has long been recognized as a powerful aid in computer architecture education [20,19], specifically through multi-cycle pipeline diagrams standard in textbooks such as [15]. Yet these paper-based representations only scale to short instruction sequences and idealized latencies.

Building on this convention, we present MinorFlow, an open-source tool that turns the MinorTrace output produced by gem5’s MinorCPU model into a multi-cycle pipeline diagram. It has two parts: a command-line tracer, which needs only a Python 3 interpreter and its standard library, and a viewer that is a single self-contained HTML file opened directly in any browser, with no installation and no further dependencies. The visual language is deliberately aligned with the diagrams already familiar to students from [15], so that the transition from the textbook pipeline to a realistic in-order core is a refinement of an existing mental model rather than a replacement of it. Beyond the basic stage grid, the tool exposes the per-instruction events behind the aggregate counters of `stats.txt`: the separation between fetch request and fetch response, inter-stage buffer occupancy, load-store queue backpressure, branch redirects, pipeline flushes, and cache hit and miss events. This lightweight workflow turns MinorFlow into an instrument for experimentation, allowing students to modify a program, re-run gem5, and immediately observe the effect on the pipeline diagram, closing a loop of hypothesis, experiment, and observation. Because both gem5 and the viewer run comfortably on commodity hardware, this loop is accessible without the FPGA boards, HDL toolchains, or dedicated lab infrastructure that hardware-based pipeline experimentation traditionally requires.

The remainder of this paper is organized as follows. Section 2 reviews related visualization tools and their limitations. Section 3 describes the gem5 configuration used to generate traces. Section 4 presents the architecture and implementation of the visualization tool. Section 5 reports functional validation through microbenchmarks and cross-validation against gem5’s built-in tools. Section 6 discusses pedagogical applications and case studies. Section 7 concludes with directions for future work.

2 Related Work

There are several tools that help students visualize the evolution of a pipeline during the execution of a program. Some can process instruction traces generated by external simulators such as gem5 [4], while others use an interactive datapath diagram to illustrate how individual microarchitectural components behave during simulation. We review them in three categories.

The first category consists of the official gem5 tools. This includes `minorview.py`, which is the only official tool that processes MinorTrace traces generated by the MinorCPU. However, `minorview`'s implementation relies on deprecated GUI (GTK/Tk) libraries used by Python 2 [6,10], making it difficult to use on modern systems. Furthermore, it is limited to a single-cycle state view, lacking the temporal, multi-cycle pipeline perspective required to analyze the instruction flow of a program. gem5 also includes `o3-pipeview.py`, which generates a multi-cycle, text-based diagram for the out-of-order O3CPU model [3] but lacks the capacity to process MinorTrace traces, and is limited by its static, text-based format, which struggles to intuitively convey complex temporal dynamics and datapath behavior at scale.

The second category comprises visualizers that display a multi-cycle pipeline stages diagram. The *de facto* standard is Konata [17], an interactive GUI-based tool, which offers pan-and-zoom capabilities, and is also capable of indicating forwarding between stages, pipeline stalls (bubbles), and several other complex microarchitectural behaviors present in modern cores. It features 47 public forks at the time of writing and has been adopted by projects such as RSD [14] and NaxRiscv [18]. However, all Konata parsers exclusively consume O3PipeView traces or the native Kanata format. After reviewing the source code and identifiable forks, none were found to support MinorTrace.

The third category consists of educational simulators focusing on the core hardware itself, with their own pipelines: Ripes [16], a graphical processor simulator and assembly editor for RISC-V, WebRISC-V [8,9], a web-based simulator, BRISC-V [2], the VRV system simulator [13], the web-based simulator for superscalar RISC-V processors [12], the FREESS simulator [7], and SonicRV [11]. All of these are self-contained RISC-V simulators featuring didactic 5-stage Patterson/Hennessy-style pipelines that do not process traces from an external simulator.

In summary, none of the existing tools simultaneously supports gem5's MinorTrace input, renders in a standard web browser without installation, and shows a multi-cycle-per-instruction diagram of the kind introduced in [15]. The tool presented in this paper is designed to fill precisely that gap.

3 Background

3.1 The gem5 Simulator and the MinorCPU Model

The gem5 project is a modular, event-driven platform for computer architecture research and education [4], providing cycle-level models of CPU microarchitec-

tures that range from simple in-order pipelines to complex out-of-order processors. Its widespread adoption in academia and industry makes it a natural choice for undergraduate courses that aim to go beyond analytical models, since students can run real programs and vary microarchitectural parameters without access to hardware. Its CPU models are also ISA-agnostic, so any tool built on their trace output applies equally across ISAs.

Among the CPU models offered by gem5, MinorFlow targets the MinorCPU, which implements a four-stage in-order pipeline (Fetch1, Fetch2, Decode, and Execute) with configurable cache hierarchies, a Load-Store Queue (LSQ), a scoreboard-based issue mechanism, and support for dynamic branch prediction. The model is highly parameterizable: the widths of inter-stage buffers, the latencies and issue rates of each functional unit, the cache sizes and associativities, and the branch predictor structure can all be adjusted independently. Crucially for our purposes, MinorCPU emits a detailed per-instruction trace at runtime (known as MinorTrace) which records the pipeline stage occupied by each instruction at every clock cycle, along with auxiliary events such as cache requests, buffer stalls, scoreboard decisions, and branch outcomes. This trace is the primary input consumed by MinorFlow.

The choice of MinorCPU as the target model for MinorFlow reflects its pedagogical position rather than any limitation of the visualization approach itself. MinorCPU occupies a productive middle ground between gem5’s simpler one-stage models and its full out-of-order O3CPU. It is substantially richer than the five-stage didactic pipeline in [15], while remaining tractable enough for undergraduate analysis.

3.2 RISC-V Configuration as a Case Study

To exercise MinorFlow in a concrete setting, and to provide a reference configuration against which the validation experiments of Section 5 are performed, we instantiate MinorCPU with a parameter set representative of a modern single-issue in-order RISC-V core. RISC-V is a natural choice of ISA for this case study for two reasons. First, the current edition of [15] adopts RISC-V as its target architecture, so students are likely to encounter the ISA in their coursework. Second, the open-source RISC-V ecosystem provides a number of well-documented industrial cores [21,1] that can serve as design references when choosing realistic microarchitectural parameters.

Building on those references, the configuration is a basic single-issue in-order 64-bit RISC-V core running at a base clock of 100 MHz, augmented with a small set of optimizations characteristic of modern processors in the same class while remaining conceptually simple enough to keep the focus on MinorFlow itself. We refer to this configuration throughout the paper as the Reference Core, whose overall organization is shown in Figure 1.

Instructions flow from left to right through a speculative front-end (Fetch1 and Fetch2) and an in-order back-end (Decode, Execute and Commit), with a decoupling FIFO at the input of each stage. The branch predictor, comprising a local Branch History Table (BHT), a Branch Target Buffer (BTB) and a Return

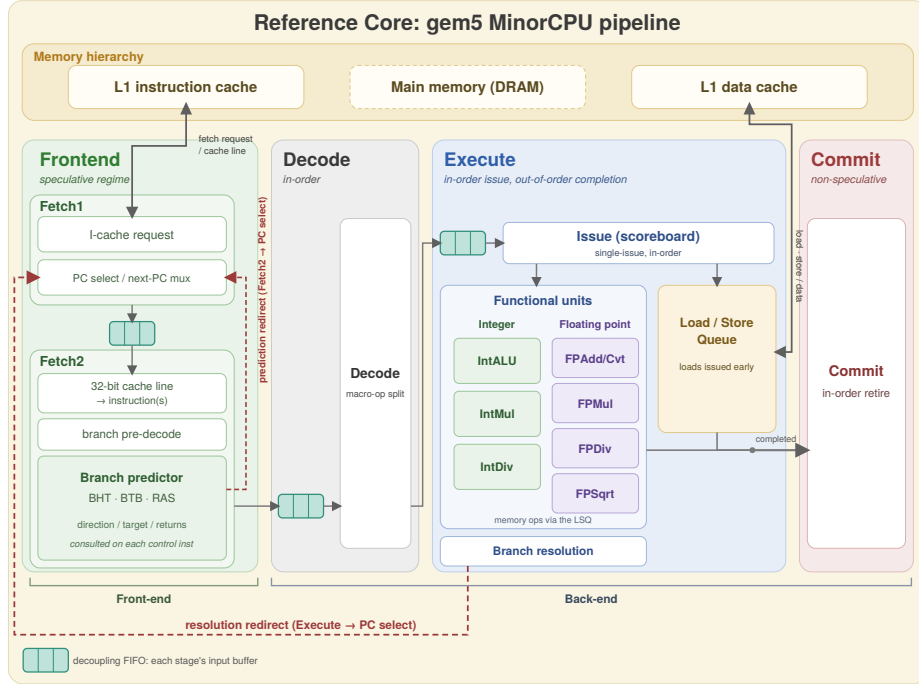


Fig. 1. Reference Core main architecture

Address Stack (RAS), sits at Fetch2, and the two red arrows in the figure mark the two ways the PC can be redirected: a prediction from Fetch2 and a resolution from Execute. The Functional units span a deliberately wide range of latencies, from a single-cycle integer ALU to a long, poorly pipelined integer divide, so instructions can finish out-of-order while Commit retires them in program order. Table 1 gives the main configuration parameters of the Reference Core that matter for the tool evaluation.

4 Tool Description

4.1 Overview and Design Rationale

MinorFlow renders a timeline diagram in which each instruction occupies a row and each pipeline stage appears as a horizontal bar segment colored by stage type. The result is a Gantt-style pipeline diagram that makes the propagation of each instruction through Fetch1, Fetch2, Decode, Execute, and Commit directly observable, along with the dwell times in the inter-stage buffers, the activity of the LSQ, and the cache responses. Commit is drawn as a stage of its own even though the MinorCPU model retires instructions from within Execute, so that the diagram matches the five-stage reading students bring from [15].

The tool is organized as two components, shown as the Visualize step of Figure 2. A command-line tracer, `MinorFlow_tracer.py`, reads the textual debug

Table 1. Configuration parameters of the Reference Core

Area	Parameter	Value
Pipeline	Fetch / Decode / Issue rate	1 instruction per cycle
	Commit rate	Up to 2 instructions per cycle
	Fetch1 to Fetch2 buffer	3 entries
	Decode input buffer	4 entries
	Functional unit issue queue	8 entries
Memory	Data bus width	64-bit
	L1 instruction cache	16 KiB, 4-way, 64 B line, 4 MSHRs of 8 targets, 1-cycle hit
	L1 data cache	32 KiB, 8-way, 64 B line, 8 MSHRs of 16 targets, 1-cycle hit
Branch prediction	Direction predictor (LocalBP)	1024-entry local history, 2-bit saturating counters
	Branch target buffer	256-entry, 4-way set-associative
	Return address stack	16 entries

trace emitted by `gem5` and converts it into a compact JSON document holding one record per instruction, together with the microarchitectural parameters it was able to recover from the trace. A viewer, `MinorFlow.html`, loads that document and draws the diagram.

Splitting the tool in two follows from the size of the input rather than from any inherent complexity of the parsing. A browser cannot hold a multi-gigabyte trace as a single string, whereas the tracer streams its input and never materializes it in memory, so a run is parsed once into a durable document that can be reopened, shared or archived without repeating the simulation.

Figure 2 shows the full workflow, from producing the trace to opening it in the viewer. A short test program is first authored in either C or RISC-V assembly. Then it is cross-compiled with `riscv64 GCC` into a static ELF binary, which `gem5` then simulates in `syscall-emulation` mode against a chosen `MinorCPU` configuration, emitting a textual trace under the requested debug flags. Only the final step leaves the cross-compilation toolchain behind: a single command converts the trace, and the resulting document is dragged onto the viewer’s drop zone and rendered immediately.

It is worth delimiting what the tool contributes. Every event the timeline shows is already emitted by `gem5`, and `MinorFlow` neither adds instrumentation nor modifies the simulator. What it adds is a representation: the events belonging to one instruction, scattered across thousands of trace lines, are joined under a single row and placed on a common cycle axis, so that a stall and the instruction that caused it become adjacent rather than merely both present in the log.

The remainder of this section examines the Visualize step of Figure 2 in detail, outlining the trace format consumed by the tool and the parsing mechanisms that drive the visualization.

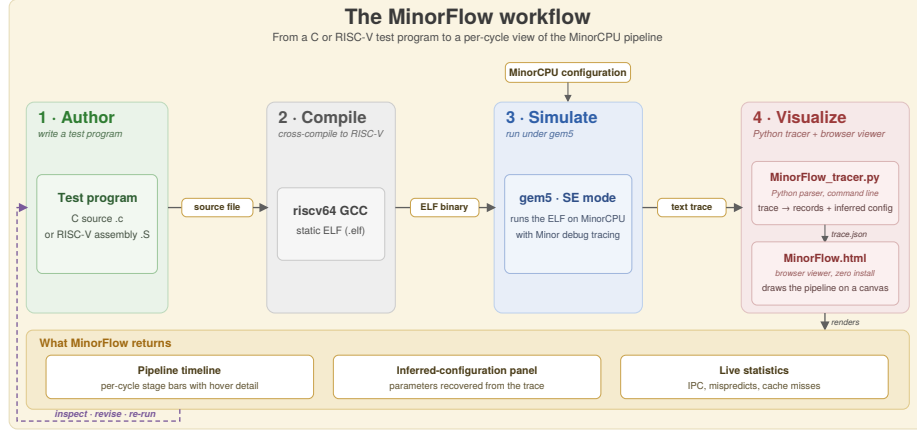


Fig. 2. The MinorFlow workflow, from a test program through gem5 simulation to the resulting timeline

4.2 Trace Format

The tool consumes the standard textual output emitted by gem5 when invoked with the debug flags shown in Listing 1.1.

Listing 1.1. Trace generation command

```
build/RISCV/gem5.opt --debug-flags=Minor,MinorTrace,MinorTiming,CacheAll,
    ExecAll,Fetch,Decode,IEW,Commit,LSQ,Scoreboard,Writeback
    minorcpu_config.py code_elf
```

Then that trace is converted into the document that the viewer consumes in a single command, shown in Listing 1.2.

Listing 1.2. Trace conversion

```
python3 MinorFlow_tracer.py trace.txt -o trace.json
```

Trace length is not bounded by the tool itself: the tracer streams its input and what the browser finally loads is the much smaller JSON document.

4.3 Tracer

The tracer is the bridge between gem5’s textual diagnostic output and the model the viewer renders. Its task is to read the trace produced by a MinorCPU run and emit, for every committed instruction, the sequence of cycles at which the instruction enters and leaves each pipeline stage, along with the auxiliary information the timeline needs to be self-explanatory.

The choice of an event-driven tracer built around regular expressions follows directly from the nature of the input. MinorTrace is not a structured data format but a stream of human-readable diagnostic messages emitted by the simulator’s debug infrastructure, with stable but informal formatting. The trace is read in

two streaming passes. The first recovers the simulated clock period, taken as the mode of the positive differences between consecutive timestamps. The second dispatches each line through a small set of patterns that recognize the events relevant to pipeline timing and assembles the per-instruction records.

Correlating events across pipeline stages is straightforward because `gem5`'s own bookkeeping carries most of the necessary information. Each instruction reported by `MinorTrace` is tagged with a composite identifier that names the cache line it was fetched from, a per-instruction sequence number that follows it through the back-end, and the speculation tags marking the stream and prediction it belongs to. The tracer uses those identifiers as keys, so that front-end and back-end events are joined under the same instance even when they appear far apart in the trace.

4.4 Renderer

The renderer is implemented in HTML5 Canvas as three coordinated panels. A fixed left column lists each instruction’s index, program counter, and mnemonic. A top ruler shows the cycle axis with tick spacing that adapts to the current zoom. The main timeline occupies the rest of the screen. The default view that opens after a document is loaded is shown in Figure 3.

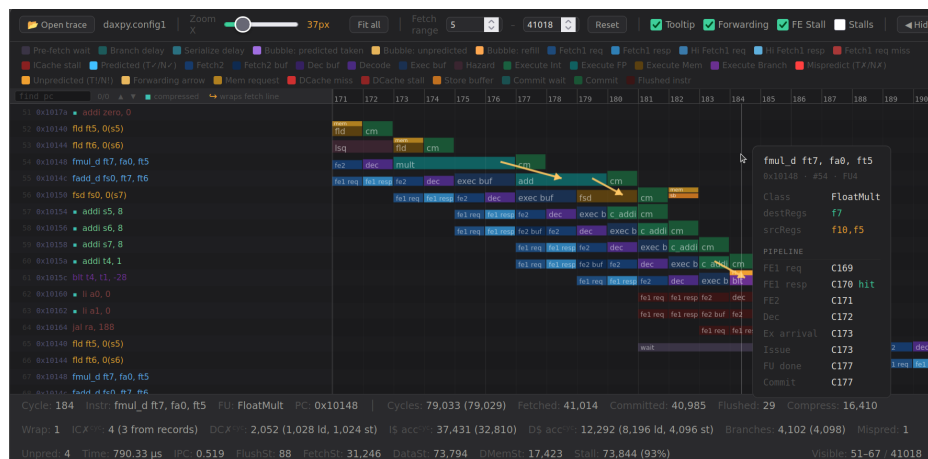


Fig. 3. Default view after loading a DAXPY trace, with the per-instruction inspector open on the right and the status bar at the bottom

Rows are ordered top to bottom by commit order and within a row the colored segments record the time spent in every stage, from fetch through the execute bar (tinted by functional unit) to the final commit marker. Events outside the strict pipeline are overlaid on the same row: cache misses extend the front-end or attach a strip over the execute bar, branch-bearing rows carry a marker indicating the prediction outcome, and instructions discarded after a redirect are drawn in red. Hovering any row brings up the per-instruction inspector visible at the right of

the figure, which reports the entry cycle of every stage, the residency in every inter-stage queue and other per-instruction diagnostics.

Reading the same view at multiple scales is one of the features the viewer is built around, and it is illustrated in Figure 4. At a low zoom level the cycle axis compresses hundreds of cycles into a single screen, so the staircase shape of a program jumps out at a glance. At a high zoom level each cell is wide enough that the stage labels are drawn inside their own bars, individual cycles can be counted off against the ruler, and the forwarding overlay draws producer-to-consumer arrows over the timeline so that data dependencies become visible.



Fig. 4. MinorFlow at two zoom levels: a big region of the program at low zoom (top) and one loop iteration at high zoom (bottom)

A small number of focus controls keep the viewer usable on traces that contain tens of thousands of instructions. A fetch-sequence range can be set to restrict the rows to an explicit window. The forwarding overlay and the per-instruction tooltip are each toggleable, so the timeline can be shown stripped of either layer when only the pipeline structure matters. A status bar at the bottom of the window updates as the visible region changes, reporting cycles, instructions fetched, committed and flushed, IPC, instruction- and data-cache accesses and misses, branch totals with mispredicted and unpredicted outcomes, and the share of cycles lost to stalls. Finally, a config panel on the toolbar exposes the MinorCPU parameters that the tracer was able to recover from the trace.

4.5 Input Validation and Error Handling

Because the tool is meant to be used by students on their own machines, both components are written to fail in a way that names the problem rather than leaving the user with an empty screen.

The tracer validates its arguments before doing any work: a missing or unreadable trace file is reported by name and the program exits with a non-zero status. The most common mistake in practice is running `gem5` without the debug flags of Listing 1.1. The tracer detects that case explicitly, since it finishes the second pass having built no records, and emits a warning naming the likely cause. Input and output errors are caught and reported rather than surfacing as a traceback.

The viewer applies the same principle to the documents it is given. A dropped file is checked in three stages: that it can be read at all, that it is syntactically valid JSON, and that it has the structure the renderer expects. The structural check verifies that the top level is an object, that the instruction array is present and non-empty, and that the first record carries every timeline field the tracer always emits, reporting the first missing field by name. Each failure raises a message naming the specific problem.

Failure recovery matters most for large inputs, where the browser rather than the tool imposes the limit. When a document exceeds the size a JavaScript engine can hold as a single string, the viewer does not simply refuse it. Because that size is dominated by the auxiliary event arrays rather than by the instruction count, a file past the byte limit may still describe few enough instructions to be rendered whole. The viewer therefore counts them in a background pass, reporting progress and remaining cancellable throughout, and then either loads the document whole or offers to load an explicit range of it.

5 Tool Evaluation

5.1 Functional Validation

To validate the viewer’s reported metrics, we cross-check them against `gem5`’s own `stats.txt` and a closed-form estimate derived from the code, on the daxpy kernel $z_i = a \cdot x_i + y_i$ for $N = 4\,096$ double-precision elements. We focus on three metrics: committed instructions, IPC and the L1 data-cache load miss count.

In our RISC-V daxpy, the inner loop commits ten instructions per iteration (two floating-point loads, one multiply, one add, one store, three pointer increments, one counter increment and one conditional branch), so the program should commit about $4\,096 \times 10 \approx 40\,960$ instructions plus a small fixed overhead. The MinorCPU pipeline issues at most one instruction per cycle, and load-use and floating-point dependency stalls roughly double the per-iteration cost, giving a predicted IPC of about $10/20 = 0.5$. With a 64-byte cache line and 8-byte doubles, the two read arrays each touch $4\,096/8 = 512$ distinct lines, giving 1 024 compulsory load misses for the inner loop. Table 2 reports the comparison and the corresponding status bar can be seen at the bottom of Figure 3.

Table 2. Cross-validation of MinorFlow against gem5 `stats.txt` and a hand estimate, on daxpy with $N = 4096$

Metric	Manual	<code>stats.txt</code>	MinorFlow
Committed instructions	$\sim 40\,960$	40 985	40 985
IPC	~ 0.50	0.519	0.519
D-cache load misses	$\sim 1\,024$	1 028	1 028

The two measured columns agree exactly, which is what the validation is after: the viewer’s timeline and gem5’s own counters describe the same execution. The hand estimate is deliberately approximate and models the inner loop alone, so the setup code that precedes it accounts for both the four additional misses and the instructions committed above the predicted 40 960.

5.2 Illustrative Case Studies

To illustrate its analytical value, this section presents three scenarios where MinorFlow captures pipeline behaviors that are invisible in standard aggregate statistics. The visualizer explicitly renders the timeline of these events, enabling a clear understanding of exactly when and why they occur.

Case Study A: Functional Unit Latency and Pipeline Backpressure

This scenario evaluates the execution of a code segment featuring a sequence of consecutive division operations. Because the MinorCPU is a strict in-order core, it is highly susceptible to structural hazards and backpressure caused by high-latency instructions. We expect the division operations to occupy the Execute stage for several cycles, generating backpressure toward the front-end of the core. As illustrated in Figure 5, the front-end accumulates instructions until the Execute, Decode, and Fetch2 queues reach their maximum hardware capacities (8, 4, and 3 instructions, respectively). This queue saturation ultimately triggers a fetch stall represented by the wait label. MinorFlow allows users to visually track the exact cycle where the Execute buffer fills to capacity, as well as the cascading effect as the Decode buffer fills, leading to the aforementioned fetch stall at cycle 139.

Case Study B: Branch Resolution and Pipeline Flushes

Speculative execution represents another critical area where aggregate metrics fall short. Figure 6 follows a taken backward branch for which the front-end held no prediction. The branch at `0x106e0` is decoded at cycle 20848 and reaches the Execute stage at cycle 20849, where it resolves as taken and is flagged as unpredicted. Because no prediction had redirected the front-end, fetching had continued down the fall-through path, and the two instructions brought in there (drawn in dark red) are discarded. Fetching restarts from the branch target at `0x100c2` at cycle 20853, after the branch delay that follows the redirect. The contrast with a predicted branch is visible in the same figure: the branch at `0x100c2` carries its flag in

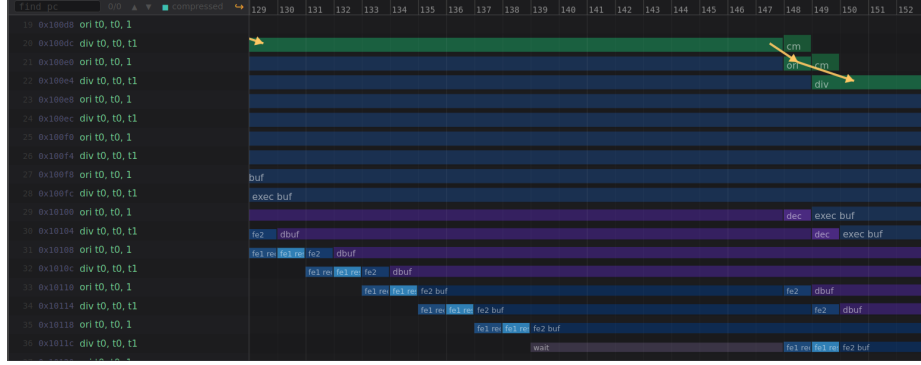


Fig. 5. Pipeline backpressure from a sequence of integer divisions (Case Study A)

the Fetch2 stage at cycle 20861, where the predictor is consulted, rather than in Execute. MinorFlow marks the two outcomes in different colors, so a flush caused by a missing prediction is distinguishable from one caused by a wrong prediction, a distinction the aggregate counters report only as separate totals.

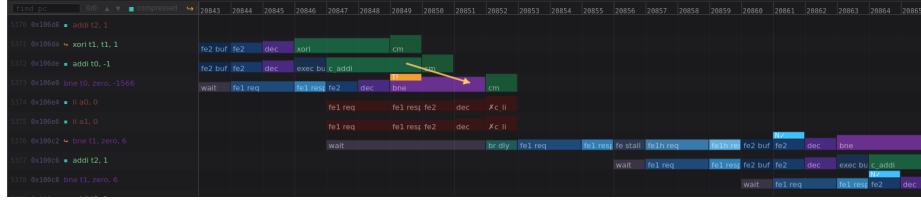


Fig. 6. An unpredicted taken branch and the resulting pipeline flush (Case Study B)

Case Study C: Memory Latency and Cache Miss Stalls In this scenario, we evaluate the pipeline’s behavior during an instruction cache miss. As shown in Figure 7, the first fetch stage, Fetch1, requests an instruction at cycle 40 but encounters a cache miss. This miss stops the front-end of the core, stalling the stage until cycle 58, when the instruction is finally retrieved from memory and normal fetch operations resume. MinorFlow provides an explicit, cycle-accurate visualization of this event, allowing users to measure the exact duration of the delay implied by an instruction cache miss.

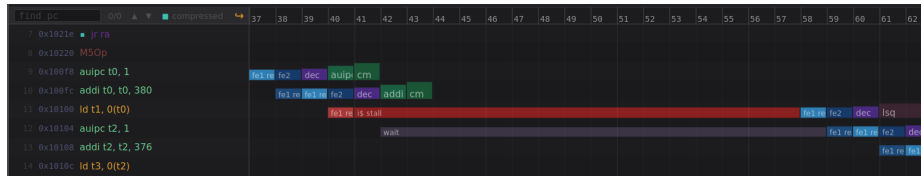


Fig. 7. Instruction cache miss stalling the front-end (Case Study C)

6 Pedagogical Context and Applicability

6.1 The Gap from the Student’s Perspective

The motivation for MinorFlow arose directly from practical challenges encountered during two undergraduate final degree projects at FaMAF: one involving the validation of an RTL RISC-V core model, and another modifying Minor-CPU’s floating-point unit. In both instances, we hit a structural wall: neither our foundational undergraduate training nor the aggregate metrics emitted by gem5 (`stats.txt`) were sufficient to explain the complex microarchitectural behaviors we were observing. While the simulator’s standard textual output accurately reported the final performance consequences, it entirely obscured the temporal events that caused them. MinorFlow was created specifically to bridge this gap from the student’s perspective, providing the visual, cycle-level evidence necessary to connect abstract theoretical concepts with the realities of modern pipeline execution.

6.2 A Concrete Curricular Context: Computer Architecture Laboratory

A more structured instance of the same gap can be found in the second laboratory assignment of the undergraduate Computer Architecture course at FaMAF-UNC. In its current form, the laboratory asks students to run a set of programs under gem5 across several microarchitectural configurations, varying the size and associativity of the data cache, and switching between local and tournament branch predictors. For each configuration, the students collect the corresponding entries of `stats.txt` (cycle counts, hit and miss rates, misprediction rates) and are expected to explain why the metrics change as the configuration changes.

The pedagogical intent of the laboratory is well established and its questions are the right ones: the students are asked to reason about the microarchitectural causes of performance variation, not merely to report which configuration is faster. The difficulty lies in the evidence available to support those explanations. For example, a student who observes that a direct-mapped cache underperforms compared to an associative cache has access to the hit and miss count, but cannot directly visualize the events that produced them. With MinorFlow, the student’s explanation can be grounded in what is visible rather than in what is plausible given the numbers.

6.3 Proposed Integration and Future Pedagogical Work

The integration of MinorFlow into the laboratory described above requires no change in the student-facing workflow beyond one additional command. Adding the debug flags of Listing 1.1 produces the trace as a side-effect of the simulation the students already run, and the conversion of Listing 1.2 turns that trace into the document the viewer reads. The student then opens the viewer in a browser,

drags that document onto the drop zone, and proceeds to construct the requested explanation against the resulting diagram.

Whether MinorFlow measurably improves student comprehension is an empirical question, and Section 7 outlines the study we plan. In the meantime, the tool remains available as an artifact that lowers the entry barrier to detailed microarchitectural analysis in gem5, and we hope its release encourages other groups to explore its use in their own settings.

7 Conclusions and Future Work

We have presented MinorFlow, an open-source tool that converts the MinorTrace output of gem5’s MinorCPU model into a multi-cycle pipeline diagram: a command-line tracer reduces the trace to a compact document, and a single-file viewer renders that document in any browser with no installation. Its visual language is deliberately aligned with the multi-cycle-per-instruction diagrams that students of computer architecture encounter in widely adopted textbooks. The functional validation confirms that the rendered timeline is consistent with the metrics gem5 itself reports, while the case studies illustrate how the tool exposes cycle-level behavior that aggregate statistics leave implicit.

Two lines of work follow from this. The first is to measure the pedagogical claim rather than assert it. We plan a controlled study in the Computer Architecture laboratory described in Section 6, comparing the explanations students produce for the same configuration sweep with and without the tool, so that the contribution can be assessed against learning outcomes and not only against the artifact itself. The second is already under way: the same multi-cycle reading discipline has been applied to the signal-level output of an HDL simulator, reconstructing the pipeline of an RTL RISC-V core from Verilator waveform dumps. Through accessible visualization, MinorFlow aims to strengthen computer architecture education and support the growing regional HPC ecosystem.

References

1. Asanović, K., Avizienis, R., Bachrach, J., Beamer, S., Biancolin, D., Celio, C., Cook, H., Dabbelt, D., Hauser, J., Izraelevitz, A., et al.: The rocket chip generator. Technical Report UCB/EECS-2016-17, EECS Department, University of California, Berkeley (2016)
2. Bandara, S., Ehret, A., Kava, D., Kinsy, M.A.: Brisc-v: An open-source architecture design space exploration toolbox. arXiv preprint arXiv:1908.09992 (2019)
3. Bardsley, A.: O3 Model. https://www.gem5.org/documentation/general_docs/cpu_models/O3CPU (2026), accessed: 2026-05-22
4. Binkert, N., Beckmann, B., Black, G., Reinhardt, S.K., Saidi, A., Basu, A., Hestness, J., Hower, D.R., Krishna, T., Sardashti, S., et al.: The gem5 simulator. ACM SIGARCH computer architecture news **39**(2), 1–7 (2011)
5. Dongarra, J., Beckman, P., Moore, T., Aerts, P., Aloisio, G., Andre, J.C., Barkai, D., Berthou, J.Y., Boku, T., Braunschweig, B., et al.: The international exascale software project roadmap. The international journal of high performance computing applications **25**(1), 3–60 (2011)

6. gem5 Project: MinorView source code: blobs.py. <https://gem5.googleusercontent.com/public/gem5/+fbaa164b6a7275760066b62fb863501c67dd2ffe/util/minorview/blobs.py> (2023), source code, commit fbaa164b6a7275760066b62fb863501c67dd2ffe; accessed July 21, 2026
7. Giorgi, R.: Freess: An educational simulator of a risc-v-inspired superscalar processor based on tomasulo’s algorithm. In: *Proceedings of the Workshop on Computer Architecture Education*. pp. 1–8 (2025)
8. Giorgi, R., Mariotti, G.: Webrisc-v: A web-based education-oriented risc-v pipeline simulation environment. In: *Proceedings of the workshop on computer architecture education*. pp. 1–6 (2019)
9. Giorgi, R., Mariotti, G.: Webrisc-v: A 64-bit risc-v pipeline simulator for computer architecture classes. *arXiv preprint arXiv:2504.03722* (2025)
10. GNOME Developer Documentation: Pygobject guide: Porting from static bindings (pygtk), <https://pygobject.gnome.org/guide/porting.html>, accessed: 2026-05-22
11. Große, D., Reiting, S., Klemmer, L.: Sonicrv: An educational platform for web-based simulation and visualization of risc-v processor architectures. *IEEE Access* (2026)
12. Jaros, J., Majer, M., Horky, J., Vavra, J.: Web-based simulator of superscalar risc-v processors. In: *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. pp. 1676–1684. *IEEE* (2024)
13. Krim, N., Porquet-Lupine, J.: Vrv: A versatile risc-v simulator for education. In: *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 2*. pp. 1505–1506 (2025)
14. Mashimo, S., Fujita, A., Matsuo, R., Akaki, S., Fukuda, A., Koizumi, T., Kado-moto, J., Irie, H., Goshima, M., Inoue, K., et al.: An open source fpga-optimized out-of-order risc-v soft processor. In: *2019 International Conference on Field-Programmable Technology (ICFPT)*. pp. 63–71. *IEEE* (2019)
15. Patterson, D., Hennessy, J.: *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. The Morgan Kaufmann Series in Computer Architecture and Design, Morgan Kaufmann (2020)
16. Petersen, M.B.: Ripes: A visual computer architecture simulator. In: *2021 ACM/IEEE Workshop on Computer Architecture Education (WCAE)*. pp. 1–8. *IEEE* (2021)
17. Shioya, D.: Konata: An instruction pipeline visualizer for Onikiri2-Kanata and Gem5-O3PipeView formats. <https://github.com/shioyadan/Konata> (2025), accessed: 2026-05-22
18. SpinalHDL: NaxRiscv core documentation. https://github.com/SpinalHDL/nax_riscv_doc (2021), accessed: 2026-05-23
19. Yeh, T.Y., Sterner, M., Bell, C., Cowe, A.C.: Visualization with experiential learning to encourage participation and research in computer architecture. In: *Proceedings of the Workshop on Computer Architecture Education*. pp. 9–16 (2023)
20. Yehezkel, C., Ben-Ari, M., Dreyfus, T.: The contribution of visualization to learning computer architecture. *Computer Science Education* **17**(2), 117–127 (2007)
21. Zaruba, F., Benini, L.: The cost of application-class processing: Energy and performance analysis of a linux-ready 1.7-ghz 64-bit risc-v core in 22-nm fdsoi technology. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* **27**(11), 2629–2640 (2019)